Probability and Statistics Lab 2: random variables, and limit theorems

General comments and instructions

- The report must be prepared as *jupyter notebook along with compiled PDF*.
- For each task, include
  - the statistics obtained (like sample mean or anything else you use to complete the task) as well as histograms etc to illustrate your findings;
  - justification of your solution (e.g. refer to the corresponding theorems from probability theory); whenever applicable, provide LaTeX formulas and derivations.
  - conclusions (e.g. how reliable your answer is, does it agree with common sense expectations etc.).
- Not complying with these instructions may result in point deduction.

Work breakdown:

- NameSurname1: ...
- NameSurname2: ...
- NameSurname3: ...

A **team id number** referred to in tasks is the **two-digit** ordinal number of your team on the list. We keep it for reproducible calculations.

```
import random
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from scipy.stats import ecdf, norm, poisson, binom, gamma

team_id: int = 33

random.seed(team_id)
np.random.seed(team_id)

z975 = norm.ppf(0.975)   # 1.96  (two-sided 95%)
z95 = norm.ppf(0.95)     # 1.645 (one-sided 95%)
```

Task 1. Hamming codes

In this task, we discuss the $[7, 4]$ Hamming code and investigate its reliability. That coding system can correct single errors in the transmission of 4-bit messages and proceeds as follows:

(a) given a message $m = (a_1 a_2 a_3 a_4)$, we first encode it to a 7-bit codeword $c = mG = (x_1 x_2 x_3 x_4 x_5 x_6 x_7)$, where G is a 4×7 generator matrix

(b) the codeword c is transmitted, and r is the received message

(c) r is checked for errors by calculating the syndrome vector $z := rH$, for a 7×3 parity-check matrix H

(d) if a single error has occurred in r, then the binary $z = (z_1 z_2 z_3)$ identifies the wrong bit no. $z_1 + 2z_2 + 4z_3$; thus (000) shows there was no error (or more than one), while (110) means the third bit (or more than one) got corrupted

(e) if the error was identified, then we flip the corresponding bit in r to get the corrected $r^* = (r_1^* r_2^* r_3^* r_4^* r_5^* r_6^* r_7^*)$; the decoded message is then $m^* := (r_3^* r_5^* r_6^* r_7^*)$.

The generator matrix G and the parity-check matrix H are given by

$$G := \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}, \qquad H^{\top} := \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

GH must equal a zero-matrix modulo 2.

Assume that each bit in the transmission $c \mapsto r$ gets corrupted independently of the others with probability $p = (2 + \frac{\text{team-id-number}}{4})/100$. Your task:

1. Simulate the encoding-transmission-decoding process N times and find the estimate $\hat{p}$ of the probability p^* of correct transmission of a single message m. Comment why, for large N, $\hat{p}$ is expected to be close to p^*.
2. By estimating the standard deviation of the corresponding indicator of success by the sample standard deviation and using the CLT, find the half-length ε of the confidence interval $(\hat{p} - \varepsilon, \hat{p} + \varepsilon)$, which contains the true value p^* with probability at least 0.95. What choice of N guarantees that $\varepsilon \le 0.03$?
3. Draw the histogram of the number $k = 0, 1, 2, 3, 4$ of errors in m^* while transmitting a 4-digit binary message. Do you think the random variable that counts the number of wrong bits in a decoded message has one of the known distributions? Justify your answer.

```
G = np.array([[1, 1, 1, 0, 0, 0, 0],
              [1, 0, 0, 1, 1, 0, 0],
              [0, 1, 0, 1, 0, 1, 0],
              [1, 1, 0, 1, 0, 0, 1]])
H = np.array([[1, 0, 1, 0, 1, 0, 1],
              [0, 1, 1, 0, 0, 1, 1],
              [0, 0, 0, 1, 1, 1, 1]]).T          # H is 7 x 3
print("G H mod 2 =\n", (G @ H) % 2)              # must be the zero matrix

p = (2 + team_id / 4) / 100
N = 100_000

m = np.random.randint(0, 2, size=(N, 4))                # random 4-bit messages
c = (m @ G) % 2                                         # encode
r = (c + (np.random.rand(N, 7) < p)) % 2                # noisy channel: flip each bit w.p. p
z = (r @ H) % 2                                         # syndrome
pos = z @ [1, 2, 4]                                     # 0 = no error found, j = bit j is wrong
bad = pos > 0
r[np.where(bad)[0], pos[bad] - 1] ^= 1                  # flip the suspected bit
m_star = r[:, [2, 4, 5, 6]]                             # decoded message (r3, r5, r6, r7)

success = np.all(m == m_star, axis=1)                   # correct transmission?
p_hat = success.mean()
p_exact = (1 - p)**7 + 7 * p * (1 - p)**6               # P(at most one flipped bit)
print(f"p = {p},  N = {N}")
print(f"p_hat = {p_hat:.4f},   exact p* = {p_exact:.4f}")
```

Output:

```
G H mod 2 =
 [[0 0 0]
 [0 0 0]
 [0 0 0]
 [0 0 0]]
p = 0.1025,  N = 100000
p_hat = 0.8441,   exact p* = 0.8441
```

Task 1.1: simulation and the law of large numbers **Setup.** For team 33, $p = (2+33/4)/100 = 0.1025$. The printed matrix GH mod 2 is the zero matrix, so G and H are consistent. The decoder follows steps (a)-(e) exactly, vectorised over $N = 100\,000$ random messages.

Result. The indicator $I_j = \mathbf{1}\{m_j^* = m_j\}$ of a correct transmission has sample mean

$$\hat{p} = \bar{I} = 0.8441.$$

Exact value (for checking). The $[7, 4]$ Hamming code is *perfect*: every 7-bit word lies within distance 1 of exactly one codeword. With at most one flipped bit the decoder recovers c. With two or more flipped bits it lands on a *different* codeword, whose message is different too. So the transmission is correct iff the number of flipped bits is ≤ 1, and that number is $\mathrm{Bin}(7, p)$:

$$p^* = (1-p)^7 + 7p(1-p)^6 = 0.8441.$$

The simulation confirms it.

Why $\hat{p} \approx p^*$. Different transmissions use independent messages and independent noise, so $I_1, \dots, I_N$ are i.i.d. Bernoulli(p^*) with $EI_j = p^*$ and finite variance. By the (weak) law of large numbers $\hat{p} \to p^*$ in probability, and by the strong law almost surely. Quantitatively, Chebyshev gives $P(|\hat{p} - p^*| \ge \delta) \le \dfrac{p^*(1-p^*)}{N\delta^2} \le \dfrac{1}{4N\delta^2}$.

Conclusion. $\hat{p} = 0.8441$ agrees with $p^* = 0.8441$ to four decimals. About 84% of the 4-bit messages arrive correctly at this noise level.

```
s = success.std(ddof=1)
eps = z975 * s / np.sqrt(N)
print(f"s = {s:.4f},  eps = {eps:.5f},  95% CI = ({p_hat - eps:.4f}, {p_hat + eps:.4f})")

# smallest N with eps <= 0.03:  N >= (z s / 0.03)^2
print("N for our s          :", int(np.ceil((z975 * s / 0.03)**2)))
print("N for any s (s <= .5):", int(np.ceil((z975 * 0.5 / 0.03)**2)))
```

Output:

```
s = 0.3628,  eps = 0.00225,  95% CI = (0.8418, 0.8463)
N for our s          : 562
N for any s (s <= .5): 1068
```

Task 1.2: confidence interval and the choice of N **Theory.** $\hat{p}$ is the mean of i.i.d. Bernoulli(p^*) variables with variance $\sigma^2 = p^*(1-p^*)$. By the CLT, $\sqrt{N}(\hat{p} - p^*)/\sigma \to \mathcal{N}(0, 1)$, so

$$P\Big(|\hat{p} - p^*| \le z_{0.975}\,\frac{\sigma}{\sqrt{N}}\Big) \to 0.95, \qquad z_{0.975} = 1.96.$$

Replacing σ by the sample standard deviation s (Slutsky's lemma) gives

$$\varepsilon = 1.96\,\frac{s}{\sqrt{N}}.$$

Numbers. $s = 0.3628$, $\varepsilon = 0.00225$, so the 95% interval is $(0.8418,\ 0.8463)$. It contains $p^* = 0.8441$.

Choice of N. The requirement $\varepsilon \le 0.03$ means $N \ge (1.96\,s/0.03)^2$. Since $s = \sqrt{p^*(1-p^*)} \le \frac{1}{2}$ for *any* p^*,

$$N \ge \Big(\tfrac{1.96 \cdot 0.5}{0.03}\Big)^2 = 1067.1 \ \Rightarrow\ N = 1068$$

is enough whatever the true probability is. For our value $s \approx 0.363$ already $N \ge 562$ suffices.

Conclusion. The estimate is very reliable for $N = 10^5$ (error about 0.2%). Roughly a thousand simulated messages are enough for ± 0.03.

```
wrong = (m != m_star)                       # N x 4 table: is decoded bit j wrong?
k = wrong.sum(axis=1)                       # number of wrong bits per message
freq = np.bincount(k, minlength=5) / N

q = wrong.mean()                            # P(a decoded bit is wrong); same for every position
binom_pmf = binom.pmf(np.arange(5), 4, q)   # what Binomial(4, q) would give
print("k | simulated | Binomial(4, q)")
for j in range(5):
    print(f"{j} | {freq[j]:.5f}   | {binom_pmf[j]:.5f}")
```

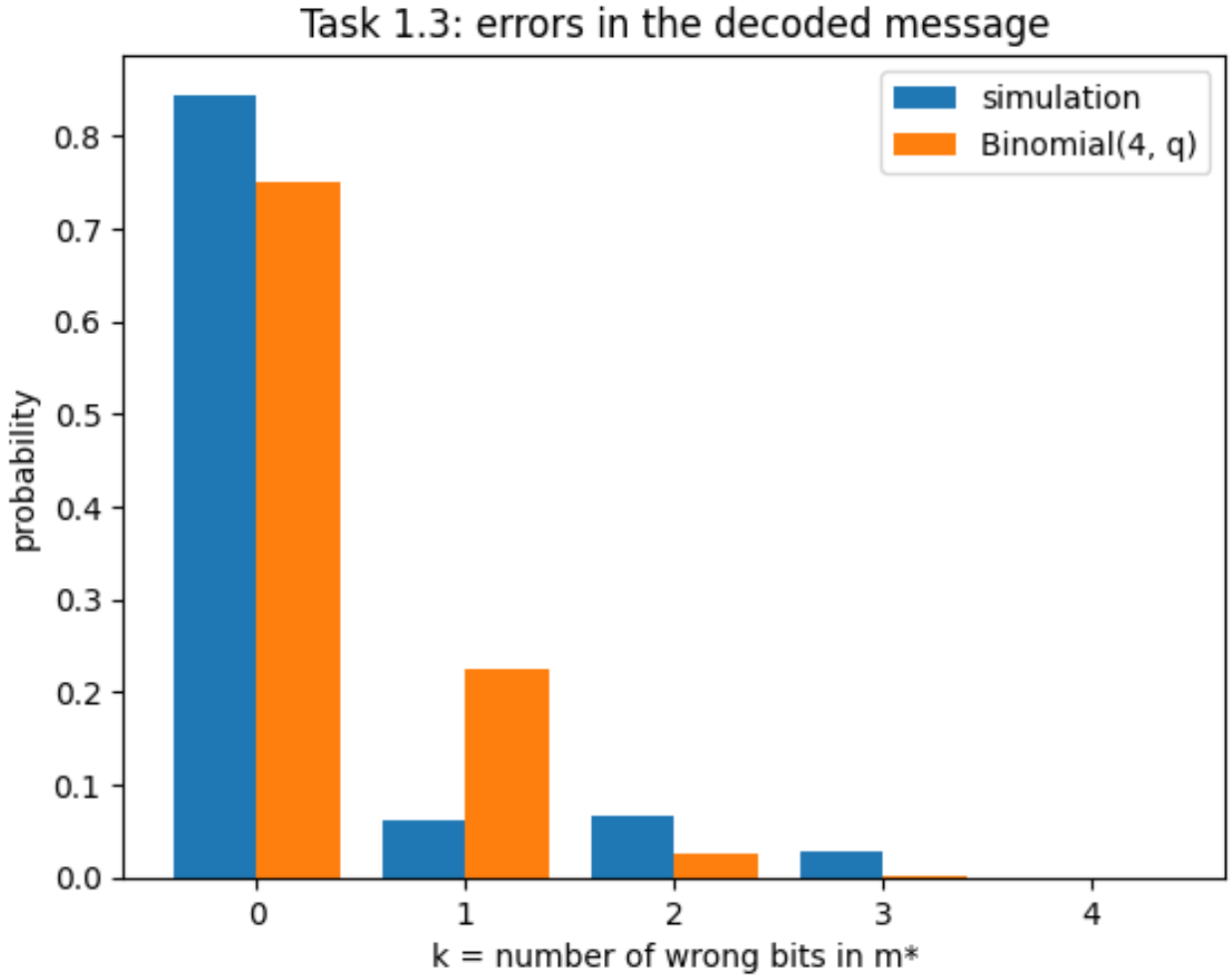
```
both = (wrong[:, 0] & wrong[:, 1]).mean()   # are bits 1 and 2 wrong independently?
print(f"\nq = {q:.4f}:  P(bit 1 and bit 2 wrong) = {both:.4f}  vs  q^2 = {q**2:.4f}")
print("P(K = k | K >= 1):", np.round(freq[1:] / freq[1:].sum(), 3))

plt.bar(np.arange(5) - 0.2, freq, width=0.4, label="simulation")
plt.bar(np.arange(5) + 0.2, binom_pmf, width=0.4, label="Binomial(4, q)")
plt.xlabel("k = number of wrong bits in m*"); plt.ylabel("probability"); plt.legend()
plt.title("Task 1.3: errors in the decoded message"); plt.show()
```

Output:

```
k | simulated | Binomial(4, q)
0 | 0.84408   | 0.74986
1 | 0.06214   | 0.22382
2 | 0.06574   | 0.02505
3 | 0.02802   | 0.00125
4 | 0.00002   | 0.00002

q = 0.0694:  P(bit 1 and bit 2 wrong) = 0.0245  vs  q^2 = 0.0048
P(K = k | K >= 1): [0.399 0.422 0.18  0.   ]
```



Task 1.3: number of wrong bits in the decoded message Let $K \in \{0, 1, 2, 3, 4\}$ be the number of wrong bits in m^*. The histogram compares the simulated frequencies with $\mathrm{Bin}(4, q)$, where $q = 0.069$ is the average probability that a decoded bit is wrong. A binomial law would require the four decoded bits to be wrong *independently*.

They are not independent. Bits 1 and 2 are wrong together with probability 0.0245, five times the

value $q^2 = 0.0048$ that independence would give. The reason is that a decoding failure (probability $1 - p^* = 0.156$, at least two channel errors) means the decoder outputs another codeword. Non-zero codewords differ from the sent one in 3, 4 or 7 positions, so a failure typically damages several message bits at once: errors come in bursts.

Consequently K is not binomial. It is a mixture: $K = 0$ with probability $p^* = 0.844$, otherwise $K \geq 1$ with conditional probabilities about (0.40, 0.42, 0.18, 0) for $k = 1, \dots, 4$. Compare with $\mathrm{Bin}(4, q)$: $P(K = 0) = 0.844$ vs 0.750, $P(K = 1) = 0.062$ vs 0.224, $P(K = 2) = 0.066$ vs 0.025, $P(K = 3) = 0.028$ vs 0.001. In particular $P(K = 2) > P(K = 1)$, which a binomial with such a small q can never produce. It is not Poisson either (bounded support).

Conclusion. K has no standard distribution.

Task 2. Particle decay

In this task, we discuss a real-life process that is well modelled by a Poisson distribution. As you remember, a Poisson random variable describes occurrences of rare events, i.e., counts the number of successes in a large number of independent random experiments. One of the typical examples is the radioactive decay process.

Consider a sample of radioactive element of mass m, which has a large half-life period T; it is vitally important to know the probability that during a one second period, the number of nuclei decays will not exceed some critical level k. This probability can easily be estimated using the fact that, given the decay constant λ of the element (i.e., the probability that a given nucleus decays within one second) and the number N of atoms in the sample, the random number of decays within a second is well modelled by Poisson distribution with parameter $\mu := N\lambda$. Next, for the sample of mass m, the number of atoms is $N = \frac{m}{M} N_A$, where $N_A = 6 \times 10^{23}$ is the Avogadro constant, and M is the molar (atomic) mass of the element. The decay constant, λ, is $\ln(2)/T$, where T is measured in seconds.

Assume that a medical laboratory receives n samples of radioactive element ^{137}Cs (used in radiotherapy) with half-life period $T = 30.1$ years and mass $m = (10 + \text{team-id-number}) \times 10^{-14}$ g each. Denote by $X_1, X_2, \dots, X_n$ the i.i.d. r.v.'s counting the number of decays in sample i in one second.

1. Specify the parameter of the Poisson distribution of X_i (you'll need the atomic mass of Cesium-137)
2. Show that the distribution of the sample means of $X_1, \dots, X_n$ gets very close to a normal one as n becomes large and identify that normal distribution. To this end,
   - simulate the realization $x_1, x_2, \dots, x_n$ of the X_i and calculate the sample mean $s = \bar{x}$;
   - repeat this K times to get the sample $\mathbf{s} = (s_1, \dots, s_K)$ of means and form the empirical cumulative distribution function $\hat{F}_{\mathbf{s}}$ of $\mathbf{s}$;
   - identify μ and σ^2 such that the c.d.f. F of $\mathcal{N}(\mu, \sigma^2)$ is close to the e.c.d.f. $\hat{F}_{\mathbf{s}}$ and plot both c.d.f.'s on one graph to visualize their proximity (use the proper scales!);
   - calculate the maximal difference between the two c.d.f.'s;
   - consider cases $n = 5$, $n = 10$, $n = 50$ and comment on the results.
3. Calculate the largest possible value of n, for which the total number of decays in one second is less than $a := 100$ with probability more than 0.95. To this end,
   - simulate the realization $x_1, x_2, \dots, x_n$ of the X_i and calculate the sum $s = x_1 + \dots + x_n$;
   - repeat this K times to get the sample $\mathbf{s} = (s_1, \dots, s_K)$ of sums;
   - calculate the number of elements of the sample which are less than critical value (a) and calculate the empirical probability;
   - obtain the theoretical bound using Markov inequality, Chernoff bound and Central Limit Theorem, and compare the results.

```
m_g = (10 + team_id) * 1e-14                # mass in grams
N_atoms = m_g / 137 * 6e23                  # molar mass of Cs-137 = 137 g/mol
T = 30.1 * 365.25 * 24 * 3600               # half-life in seconds
lam = np.log(2) / T
mu = N_atoms * lam
print(f"N = {N_atoms:.4e} atoms,  lambda = {lam:.4e} 1/s,  mu = {mu:.4f}")
```

Output:

```
N = 1.8832e+09 atoms,  lambda = 7.2972e-10 1/s,  mu = 1.3742
```

Task 2.1: the Poisson parameter Each X_i is Poisson(μ) with $\mu = N\lambda$ (a huge number N of atoms, each with a tiny decay probability λ per second, which is the Poisson limit theorem). With $M(^{137}\mathrm{Cs}) = 137$ g/mol, $m = (10+33)\cdot 10^{-14} = 4.3\cdot 10^{-13}$ g and $T = 30.1$ years $= 9.50\cdot 10^{8}$ s:

$$N = \frac{m}{M}N_A = 1.883\cdot 10^{9}, \qquad \lambda = \frac{\ln 2}{T} = 7.297\cdot 10^{-10}\ \mathrm{s}^{-1}, \qquad \boxed{\mu = N\lambda = 1.3742}.$$

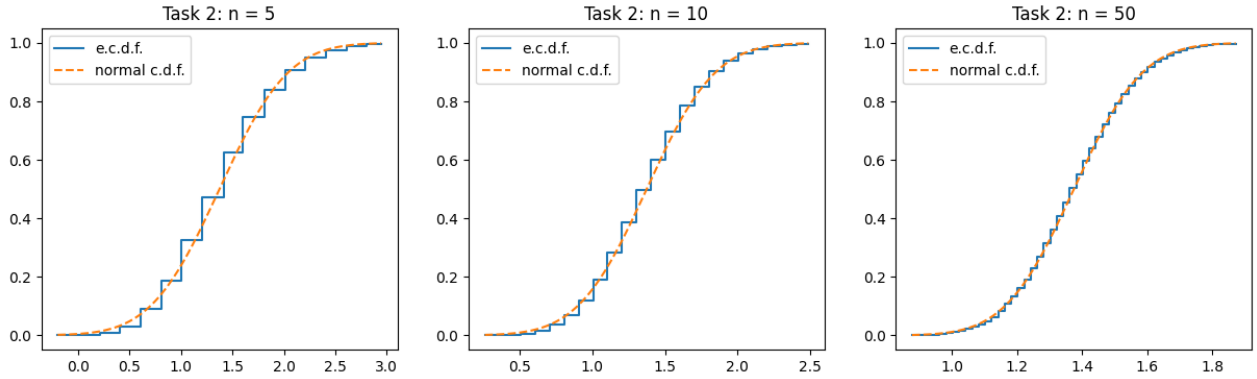
So one sample produces on average about 1.37 decays per second, and $EX_i = \mathrm{Var}X_i = \mu$.

```python
def compare_with_normal(s, mu_n, sd_n, ax, title):
    """plot e.c.d.f. of s against the N(mu_n, sd_n^2) c.d.f.; return the max difference on a 200-point grid"""
    F = ecdf(s).cdf.evaluate
    t = np.linspace(mu_n - 3 * sd_n, mu_n + 3 * sd_n, 200)
    ax.step(t, F(t), where="post", label="e.c.d.f.")
    ax.plot(t, norm.cdf(t, mu_n, sd_n), "--", label="normal c.d.f.")
    ax.set_title(title); ax.legend()
    return np.max(np.abs(F(t) - norm.cdf(t, mu_n, sd_n)))

K = 10_000
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
print(" n | mean(s) | var(s)  | mu/n    | max diff | diff*sqrt(n) | half largest jump")
for ax, n in zip(axes, (5, 10, 50)):
    s = np.random.poisson(mu, size=(K, n)).mean(axis=1)       # K sample means
    d = compare_with_normal(s, mu, np.sqrt(mu / n), ax, f"Task 2: n = {n}")
    jump = 0.5 * poisson.pmf(int(n * mu), n * mu)             # lattice effect
    print(f"{n:2d} | {s.mean():.4f}  | {s.var(ddof=1):.5f} | {mu/n:.5f} | {d:.4f}   | {d*np.sqrt(n):.3f}
    ↪  | {jump:.4f}")
plt.show()
```

Output:

```
 n | mean(s) | var(s)  | mu/n    | max diff | diff*sqrt(n) | half largest jump
 5 | 1.3726  | 0.27858 | 0.27484 | 0.0977   | 0.219        | 0.0758
10 | 1.3702  | 0.13166 | 0.13742 | 0.0737   | 0.233        | 0.0539
50 | 1.3743  | 0.02759 | 0.02748 | 0.0326   | 0.231        | 0.0241
```



Task 2.2: sample means are approximately normal **Theory.** The X_i are i.i.d. with mean μ and variance μ, so the CLT gives

$$\bar{X}_n \approx \mathcal{N}\Big(\mu,\ \frac{\mu}{n}\Big), \qquad \mu = 1.3742.$$

(Indeed $n\bar{X}_n \sim \mathrm{Poisson}(n\mu)$ exactly.) I simulated $K = 10000$ sample means for each n and compared their e.c.d.f. with this normal c.d.f. on a grid of 200 points in $\mu \pm 3\sigma$.

Results.

n	mean of **s**	variance of **s**	μ/n	max difference
5	1.3726	0.2786	0.2748	0.0977
10	1.3702	0.1317	0.1374	0.0737
50	1.3743	0.0276	0.0275	0.0326

Comments.

- The sample moments agree with μ and μ/n, and the e.c.d.f. approaches the normal c.d.f. as n grows.
- The maximal difference decreases like $1/\sqrt{n}$ ($\max\text{diff} \cdot \sqrt{n} \approx 0.22$ is nearly constant), as predicted by the Berry-Esseen theorem.
- For a *discrete* distribution part of the error is unavoidable. $\bar{X}_n$ takes only the values k/n, so its c.d.f. jumps, while the normal c.d.f. is continuous. Half of the largest jump is 0.076, 0.054, 0.024 for $n = 5, 10, 50$, which already accounts for most of the observed differences. The rest is skewness plus Monte Carlo noise (about ± 0.014 for $K = 10000$).

Conclusion. The normal approximation is rough for $n = 5$ (error about 0.10), acceptable for $n = 10$ and good for $n = 50$ (error about 0.03).

```
a = 100
n = np.arange(1, 100)
lam_sum = n * mu                                        # S = X_1 + ... + X_n ~ Poisson(n mu)

exact = poisson.cdf(a - 1, lam_sum)                     # P(S < 100)
n_exact = n[exact > 0.95].max()
n_markov = int(0.05 * a / mu)                           # n mu / a <= 0.05
n_chernoff = n[(lam_sum < a) & (-lam_sum + a * (1 + np.log(lam_sum / a)) <= np.log(0.05))].max()
n_clt = n[(a - lam_sum) / np.sqrt(lam_sum) >= z95].max()
print(f"largest n:  Markov {n_markov},  Chernoff {n_chernoff},  CLT {n_clt},  exact {n_exact}")

# Monte Carlo check around the answer
K = 20_000
print(" n | simulated P(S<100) | exact")
for n_try in (n_chernoff, n_clt, n_clt + 1):
    S = np.random.poisson(mu, size=(K, n_try)).sum(axis=1)
    print(f"{n_try:2d} | {(S < a).mean():.4f}             | {exact[n_try - 1]:.4f}")
```

Output:

```
largest n:  Markov 3,  Chernoff 56,  CLT 61,  exact 61
 n | simulated P(S<100) | exact
56 | 0.9940             | 0.9934
61 | 0.9534             | 0.9535
62 | 0.9366             | 0.9365
```

Task 2.3: how many samples may the laboratory accept? $S = X_1 + \cdots + X_n \sim \text{Poisson}(n\mu)$. We need the largest n with $P(S < 100) > 0.95$, i.e. $P(S \geq 100) < 0.05$.

- **Markov:** $P(S \geq a) \leq \dfrac{ES}{a} = \dfrac{n\mu}{100} \leq 0.05 \Rightarrow n \leq \dfrac{5}{\mu} = 3.64$, so $n = 3$.
- **Chernoff:** $P(S \geq a) \leq \inf_{t>0} e^{-ta} E e^{tS} = \inf_{t>0} e^{-ta + n\mu(e^t - 1)}$. The optimum is $t = \ln \frac{a}{n\mu}$ (valid for $n\mu < a$) and gives $e^{-n\mu}\left(\frac{e\, n\mu}{a}\right)^a \leq 0.05$, so $n \leq 56$.
- **CLT:** $S \approx \mathcal{N}(n\mu, n\mu)$ gives $P(S < 100) \approx \Phi\left(\dfrac{100 - n\mu}{\sqrt{n\mu}}\right) \geq 0.95 \iff n\mu + 1.645\sqrt{n\mu} \leq 100 \iff n\mu \leq 84.8$, so $n \leq 61$.
- **Exact / Monte Carlo** ($K = 20000$): the exact Poisson c.d.f. gives $n = 61$, and the simulation agrees: $P(S < 100) = 0.9535$ (simulated 0.9534) for $n = 61$ and 0.9365 (simulated 0.9366) for $n = 62$, the first value below 0.95.

Comparison. Markov is extremely conservative (3 vs 61) because it uses only the mean. Chernoff uses the whole moment generating function and is much better (56) but still safe-side. The CLT

answer (61) coincides with the exact one because S is a sum of many terms and Poisson(≈ 84) is nearly symmetric.

Conclusion. At most $\boxed{n = 61}$ samples can be accepted. The Markov and Chernoff bounds are guaranteed but wasteful, while the CLT is accurate here but not a guarantee.

Task 3. Geiger counter

In this task, we use the Central Limit Theorem approximation for continuous random variables.

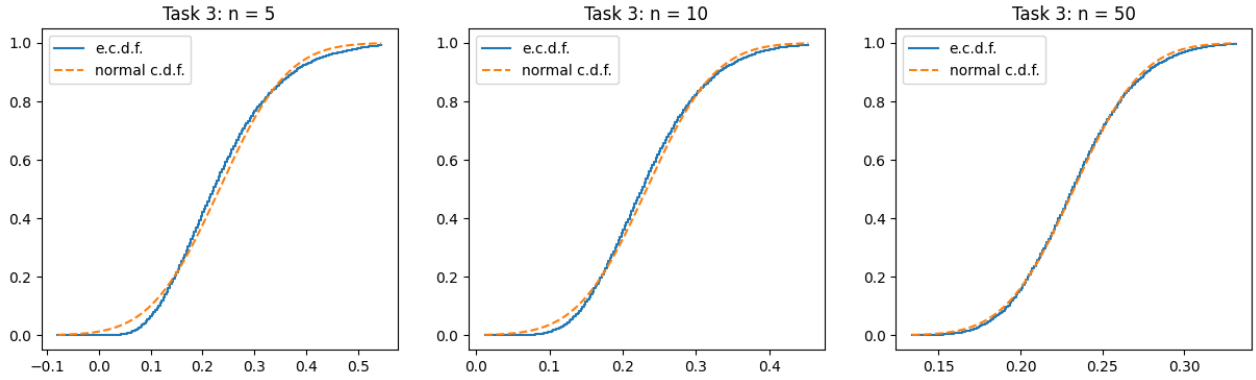
One of the devices to measure radioactivity level at a given location is the Geiger counter. When the radioactive level is almost constant, the time between two consecutive clicks of the Geiger counter is an exponentially distributed random variable with parameter $\nu_1 = \frac{\text{team-id-number}}{10} + 1$ (minutes^{-1}). Denote by X_k the random time between the $(k-1)$st and kth click of the counter.

1. Show that the distribution of the sample means of $X_1, X_2, \dots, X_n$ gets very close to a normal one (which one?) as n becomes large. To this end,
   - simulate the realizations $x_1, x_2, \dots, x_n$ of the r.v. X_i and calculate the sample mean $s = \bar{x}$;
   - repeat this K times to get the sample $\mathbf{s} = (s_1, \dots, s_K)$ of means and then the empirical cumulative distribution function $F_{\mathbf{s}}$ of $\mathbf{s}$;
   - identify μ and σ^2 such that the c.d.f. of $\mathcal{N}(\mu, \sigma^2)$ is close to the e.c.d.f. $F_{\mathbf{s}}$ and plot both c.d.f.'s on one graph to visualize their proximity;
   - calculate the maximal difference between the two c.d.f.'s;
   - consider cases $n = 5$, $n = 10$, $n = 50$ and comment on the results.
2. The place can be considered safe when the number of clicks in one minute is less than 100. It is known that the parameter ν is proportional to the number N of the radioactive samples, i.e., $\nu = N\nu_1$. Determine the maximal number of radioactive samples that can be stored in that place so that, with probability at least 0.95, the place is identified as safe. To do this,
   (a) express the event of interest in terms of the r.v. $S := X_1 + \cdots + X_{100}$;
   (b) obtain the theoretical bounds on N using the Markov inequality, Chernoff bound and Central Limit Theorem and compare the results;
   (c) with the predicted N and thus ν, simulate the realization $x_1, x_2, \dots, x_{100}$ of the X_i and of the sum $S = X_1 + \cdots + X_{100}$;
   (d) repeat this K times to get the sample $\mathbf{s} = (s_1, \dots, s_K)$ of total times until the 100th click;
   (e) estimate the probability that the location is identified as safe and compare to the desired level 0.95

```
nu1 = team_id / 10 + 1
K = 10_000
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
print(" n | mean(s) | 1/nu1  | var(s)   | 1/(n nu1^2) | max diff | diff*sqrt(n)")
for ax, n in zip(axes, (5, 10, 50)):
    s = np.random.exponential(1 / nu1, size=(K, n)).mean(axis=1)
    d = compare_with_normal(s, 1 / nu1, 1 / (nu1 * np.sqrt(n)), ax, f"Task 3: n = {n}")
    print(f"{n:2d} | {s.mean():.4f}  | {1/nu1:.4f} | {s.var(ddof=1):.6f} | {1/(n*nu1**2):.6f}    | {d:.4f}   |
     ↪  {d*np.sqrt(n):.3f}")
plt.show()
```

Output:

```
 n | mean(s) | 1/nu1  | var(s)   | 1/(n nu1^2) | max diff | diff*sqrt(n)
 5 | 0.2344  | 0.2326 | 0.010882 | 0.010817    | 0.0511   | 0.114
10 | 0.2328  | 0.2326 | 0.005376 | 0.005408    | 0.0414   | 0.131
50 | 0.2330  | 0.2326 | 0.001108 | 0.001082    | 0.0165   | 0.116
```



Task 3.1: sample means of exponential waiting times $X_i \sim \mathrm{Exp}(\nu_1)$ with $\nu_1 = 33/10 + 1 = 4.3$ min^{-1}, so $EX_i = 1/\nu_1 = 0.2326$ min and $\mathrm{Var}X_i = 1/\nu_1^2$. By the CLT

$$\bar{X}_n \approx \mathcal{N}\Big(\frac{1}{\nu_1},\ \frac{1}{n\nu_1^2}\Big).$$

n	mean of **s**	variance of **s**	$1/(n\nu_1^2)$	max difference
5	0.2344	0.010882	0.010817	0.0511
10	0.2328	0.005376	0.005408	0.0414
50	0.2330	0.001108	0.001082	0.0165

Comments. The sample moments match the theoretical ones, and the e.c.d.f. gets closer to the normal c.d.f. as n grows. There is no lattice effect here, so the whole error comes from the skewness $\gamma = 2$ of the exponential law. The first Edgeworth term predicts $\sup_x |F_n - \Phi| \approx \frac{\gamma}{6\sqrt{n}} \max_x |(x^2 - 1)\varphi(x)| = \frac{0.133}{\sqrt{n}}$, i.e. 0.060, 0.042, 0.019. This matches the observed 0.051, 0.041, 0.017 within the Monte Carlo noise of about 0.014.

Conclusion. The normal approximation is already decent for $n = 10$ and good for $n = 50$. The skewed exponential law needs a few more terms than a symmetric one, and the error decays like $1/\sqrt{n}$.

```
N = np.arange(1, 40)
nu = N * nu1                                  # S = X_1 + ... + X_100 ~ Gamma(100, nu); safe iff S > 1
exact = poisson.cdf(99, nu)                   # P(S > 1) = P(Poisson(nu) <= 99)

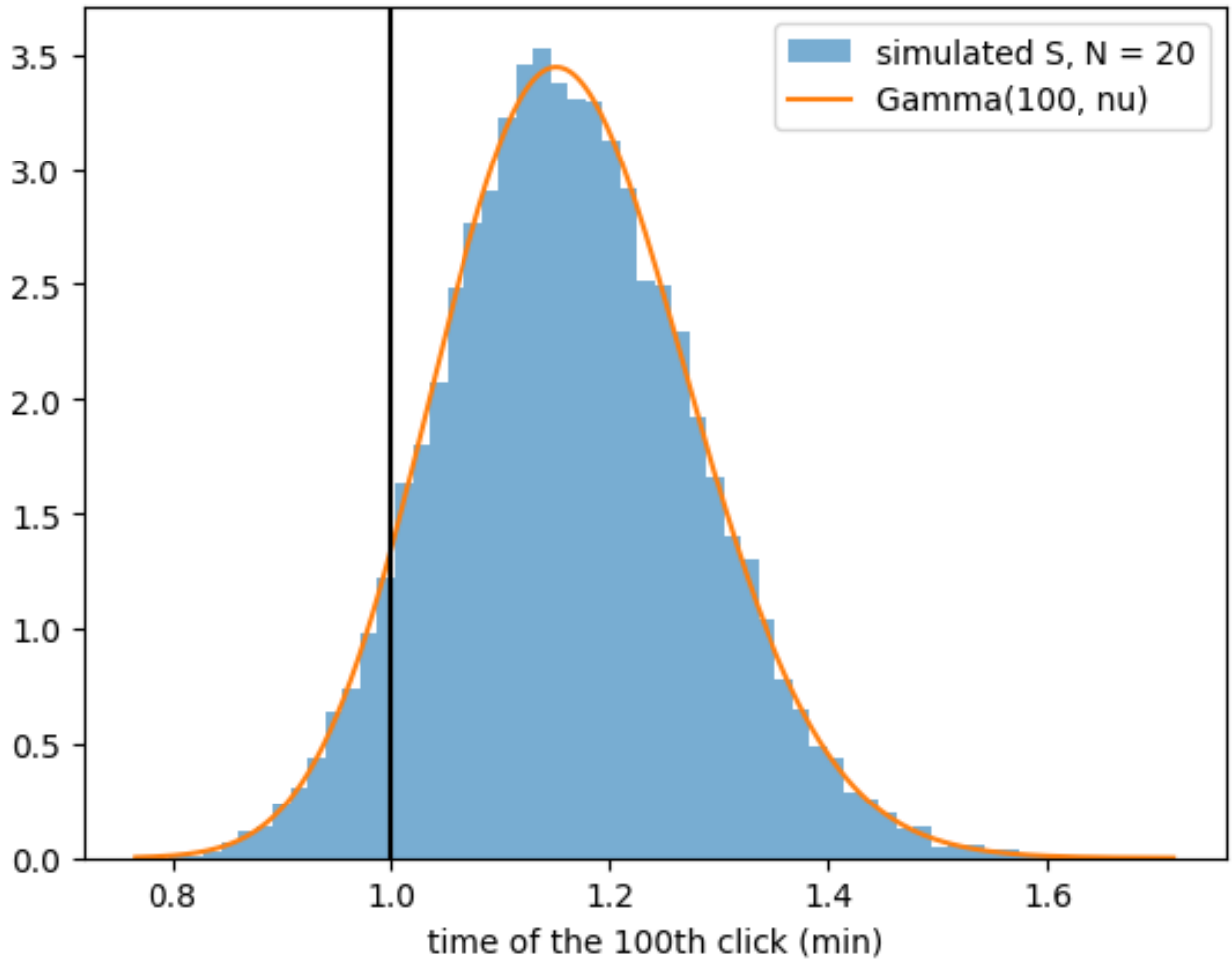
N_markov = N[nu / 99 <= 0.05].max()                                          # E[1/S] = nu/99
N_chernoff = N[(nu < 100) & (100 * np.log(nu / 100) + 100 - nu <= np.log(0.05))].max()
N_clt = N[(100 - nu) / 10 >= z95].max()                                      # Phi((100-nu)/10) >= 0.95
N_exact = N[exact >= 0.95].max()
print(f"max N:  Markov {N_markov},  Chernoff {N_chernoff},  CLT {N_clt},  exact {N_exact}")

K = 20_000
print("  N | simulated P(safe) | exact")
for N_try in (N_markov, N_chernoff, N_clt, N_clt + 1):
    S = np.random.exponential(1 / (N_try * nu1), size=(K, 100)).sum(axis=1)   # time of the 100th click
    print(f"{N_try:3d} | {(S > 1).mean():.4f}            | {exact[N_try - 1]:.4f}")

plt.hist(S, bins=60, density=True, alpha=0.6, label=f"simulated S, N = {N_clt + 1}")   # last S: N = 20
t = np.linspace(S.min(), S.max(), 300)
plt.plot(t, gamma.pdf(t, a=100, scale=1 / ((N_clt + 1) * nu1)), label="Gamma(100, nu)")
plt.axvline(1, color="k"); plt.xlabel("time of the 100th click (min)"); plt.legend(); plt.show()
```

Output:

```
max N:  Markov 1,  Chernoff 18,  CLT 19,  exact 19
  N | simulated P(safe) | exact
  1 | 1.0000            | 1.0000
 18 | 0.9925            | 0.9923
 19 | 0.9736            | 0.9727
 20 | 0.9260            | 0.9247
```



Task 3.2: how many radioactive samples can be stored? **(a) The event.** With $\nu = N\nu_1$ the waiting times are i.i.d. $\text{Exp}(\nu)$, so $S = X_1 + \cdots + X_{100} \sim \text{Gamma}(100, \nu)$ is the time of the 100th click. Fewer than 100 clicks in the first minute means the 100th click comes after time 1:

$$\text{safe} = \{S > 1\}, \qquad \text{need } P(S > 1) \geq 0.95 \iff P(S \leq 1) \leq 0.05.$$

Equivalently, the number of clicks in a minute is $\text{Poisson}(\nu)$ and we need $P(\text{Poisson}(\nu) \leq 99) \geq 0.95$. The code uses this Poisson form for the exact values.

(b) Bounds on N.

- **Markov**, applied to the non-negative variable $1/S$: $P(S \leq 1) = P(1/S \geq 1) \leq E[1/S] = \dfrac{\nu}{99}$ (since $\int_0^\infty \frac{1}{s} \frac{\nu^{100} s^{99} e^{-\nu s}}{\Gamma(100)} ds = \frac{\nu\,\Gamma(99)}{\Gamma(100)}$). Hence $\nu \leq 4.95$, $N \leq 1.15$, so $N = 1$.
- **Chernoff:** $P(S \leq 1) \leq \inf_{t>0} e^t E e^{-tS} = \inf_{t>0} e^t \left(\dfrac{\nu}{\nu + t}\right)^{100}$. The optimum is $t = 100 - \nu$ and gives $\left(\dfrac{\nu}{100}\right)^{100} e^{100-\nu} \leq 0.05$, i.e. $\nu \leq 77.48$, $N \leq 18.02$, so $N = 18$.
- **CLT:** $S \approx \mathcal{N}(100/\nu,\, 100/\nu^2)$, so $P(S > 1) \approx \Phi\left(\dfrac{100 - \nu}{10}\right) \geq 0.95 \iff \nu \leq 100 - 16.45 = 83.55$, $N \leq 19.43$, so $N = 19$.

- **Exact:** $\nu \le 84.14$, $N \le 19.57$, so $N = 19$.

(c)-(e) Simulation. For each predicted N I simulated $K = 20000$ values of $S = X_1 + \cdots + X_{100}$ and estimated $P(S > 1)$:

N	ν	simulated P(safe)	exact
1 (Markov)	4.3	1.0000	1.0000
18 (Chernoff)	77.4	0.9925	0.9923
19 (CLT)	81.7	0.9736	0.9727
20	86.0	0.9260	0.9247

The histogram (shown for $N = 20$) follows the Gamma density, and the threshold $S = 1$ cuts off the left tail, which is about 7.5% of the mass.

Conclusion. The maximal number of samples is $\boxed{N = 19}$: the place is judged safe with probability $0.973 \ge 0.95$, whereas $N = 20$ gives only 0.925. Markov (1) is useless in practice, Chernoff (18) is guaranteed and nearly sharp, and the CLT (19) gives the right answer here because S has 100 terms.

Task 4. Expectation fallacy

This task consists of two parts:

1. In this part, we discuss independence of random variables and its moments: expectation and variance.
   (a) Suppose we have a random variable X. Explain why $E\left(\frac{1}{X}\right) \neq \frac{1}{E(X)}$;
   (b) Let $X \sim \mathcal{N}(\mu, \sigma^2)$ with $\mu = 5$ and $\sigma^2 = 10$. Simulate realizations $x_1, x_2, \ldots, x_{100}$ of X and $y_1, y_2, \ldots, y_{100}$ of $Y := \frac{1}{X}$ to calculate the values of $\frac{1}{\bar{X}}$ and $\bar{Y}$. Comment on the received results;
   (c) Let X and Y be independent exponentially distributed r.v.'s with parameter $\lambda = 2$. Set $Z := \ln X + 5$. Plot the Quantile-Quantile plot and scatterplot of X and Y. Plot the Quantile-Quantile plot and scatterplot of X and Z. Explain the results. Comment on the difference of relations between the pairs of random variables. Which pair of r.v.'s is dependent and which one is identically distributed?
2. You toss a fair coin three times and a random variable X records how many times the coin shows Heads. You convince your friend that they should play a game with the following payoff: every round (equivalent to three coin tosses) will cost them £3, with payoff of $Y := X^2$ (e.g., £4 for two Heads). Your friend argues: "on average there are 1.5 Heads, so I win £$1.5^2 = $ £2.25 < £3, hence the game is unfair". Is the game fair, i.e., is the expected value of the net gain zero? What is the variance of Y? To answer this,
   (a) Explain what type of random variable is X:
      - Normally distributed
      - Binomially distributed
      - Poisson distributed
      - Uniformly distributed
   (b) What are the expected value and variance of X? Simulate realizations $x_1, x_2, \ldots, x_{100}$ of X to calculate the values of sample mean $\bar{X}$ and sample variance $s^2 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}$. Comment on the results;
   (c) Is your friend's argument correct? Relate it to part 1(a). Compute the expected value and variance of Y exactly. Simulate realizations $y_1, y_2, \ldots, y_{100}$ of Y to calculate the values of sample mean $\bar{Y}$ and sample variance $s^2 = \frac{\sum_{i=1}^{n}(y_i - \bar{y})^2}{n-1}$. Comment on the results;

(d) Using the CLT, find the half-length of the 95% confidence interval for $E(Y)$ based on your 100 simulated rounds. Could the simulation alone tell you whether the game is fair? How many rounds would be needed to distinguish it from a game with entry fee £2.9?

Task 4, part 1(a): why $E(1/X) \neq 1/E(X)$ Expectation commutes only with *linear* maps: $E(aX + b) = aEX + b$. The map $x \mapsto 1/x$ is not linear, so there is no reason for $E(1/X)$ and $1/E(X)$ to agree. For $X > 0$ non-degenerate, $x \mapsto 1/x$ is strictly convex and Jensen's inequality gives $E(1/X) > 1/E(X)$. Example: $X \in \{1, 3\}$ with probability $\frac{1}{2}$ each gives $E(1/X) = \frac{1}{2}(1 + \frac{1}{3}) = \frac{2}{3} > \frac{1}{2} = 1/EX$. If X can take values near 0 or of both signs, $E(1/X)$ may not even exist.

```
x = np.random.normal(5, np.sqrt(10), size=100)
print(f"1/mean(x) = {1/x.mean():.4f},   mean(1/x) = {(1/x).mean():.4f}")
print(f"negative x: {(x < 0).sum()},   smallest |x| = {np.abs(x).min():.4f}")

xs = np.random.normal(5, np.sqrt(10), size=(2000, 100))       # 2000 repetitions of the experiment
qs = [0, 0.05, 0.5, 0.95, 1]
print("quantiles (0, 5%, 50%, 95%, 100%) over 2000 repetitions:")
print("  1/mean(x):", np.round(np.quantile(1 / xs.mean(axis=1), qs), 3))
print("  mean(1/x):", np.round(np.quantile((1 / xs).mean(axis=1), qs), 3))
```

Output:

```
1/mean(x) = 0.1851,   mean(1/x) = 0.2466
negative x: 3,   smallest |x| = 0.2030
quantiles (0, 5%, 50%, 95%, 100%) over 2000 repetitions:
  1/mean(x): [0.165 0.182 0.2   0.224 0.257]
  mean(1/x): [-1.45667e+02 -4.22000e-01  2.32000e-01  9.67000e-01  3.42805e+02]
```

Task 4, part 1(b): $1/\bar{X}$ versus $\bar{Y}$ for $X \sim \mathcal{N}(5, 10)$ **Results (100 draws).** $1/\bar{x} = 0.1851$ is close to $1/EX = 0.2$, and $\bar{y} = 0.2466$ is of the same order. This particular run happens to look harmless (three of the 100 draws were negative, since $P(X < 0) = 0.057$), but that is luck.

Explanation. X has a positive density at 0, so $E|1/X| = \int \frac{1}{|x|} f(x)\, dx$ diverges like $\int dx/|x|$: $E(1/X)$ **does not exist**, and the law of large numbers does not apply to $\bar{Y}$. By contrast $\bar{X} \to 5$, and by continuity of $x \mapsto 1/x$ at 5 we get $1/\bar{X} \to 0.2$.

Repetition (2000 experiments of 100 draws). $1/\bar{X}$ is very stable: its 5%-95% range is 0.182-0.224 and even the extremes are only 0.165 and 0.257. $\bar{Y}$ is wildly variable: the 5%-95% range is -0.42 to 0.97, and the extremes are -146 and $+343$. This is heavy-tailed behaviour, where a single draw very close to 0 dominates the mean.

Conclusion. $1/\bar{X} \neq \bar{Y}$, and the second quantity is meaningless as an estimate of anything: "the expectation of the inverse" is not "the inverse of the expectation", and sometimes it does not exist at all.

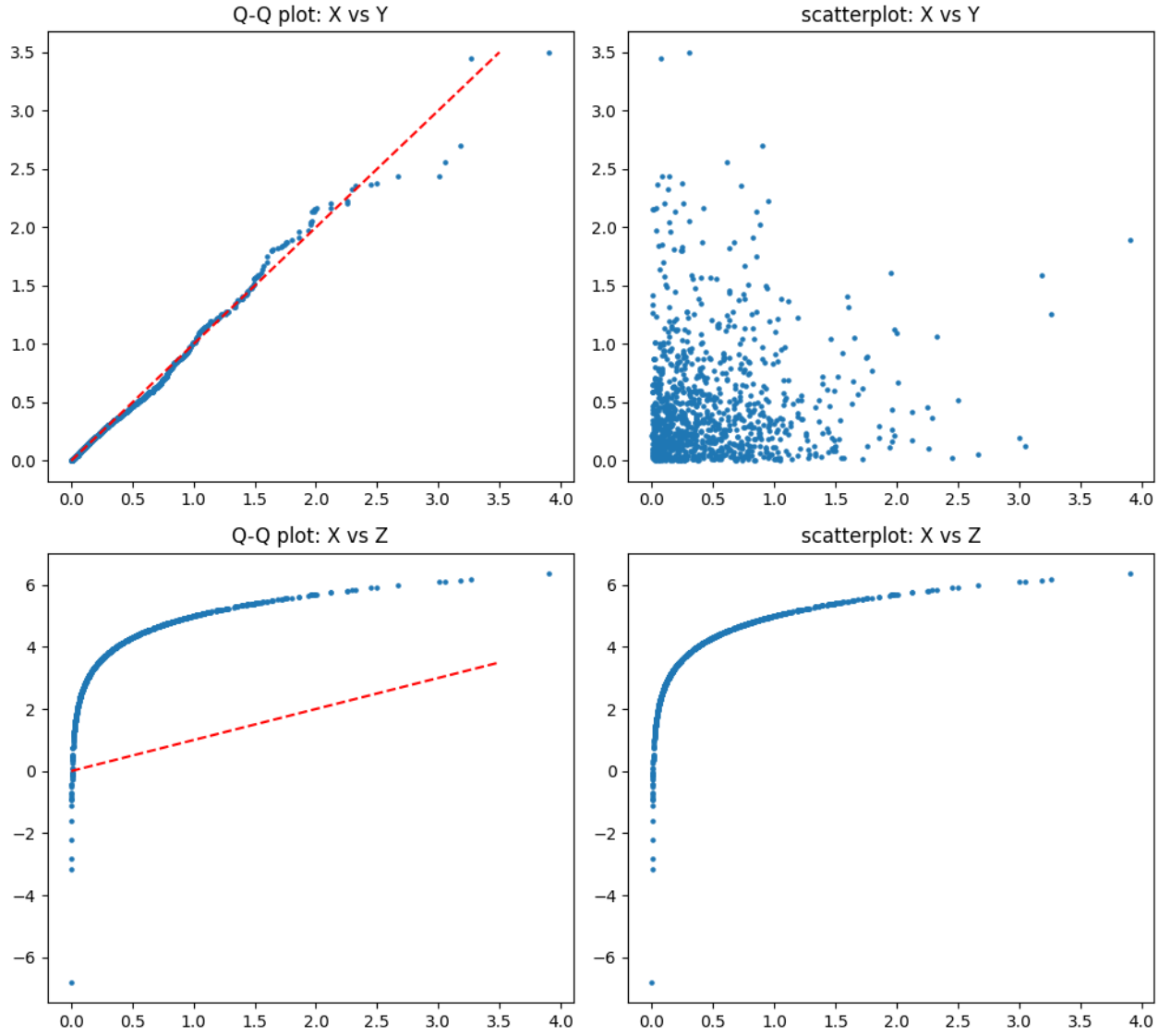
```
n = 1000
x = np.random.exponential(1 / 2, size=n)
y = np.random.exponential(1 / 2, size=n)
z = np.log(x) + 5

fig, ax = plt.subplots(2, 2, figsize=(10, 9))
ax[0, 0].scatter(np.sort(x), np.sort(y), s=5); ax[0, 0].plot([0, 3.5], [0, 3.5], "r--"); ax[0,
 ↪  0].set_title("Q-Q plot: X vs Y")
ax[0, 1].scatter(x, y, s=5);                   ax[0, 1].set_title("scatterplot: X vs Y")
ax[1, 0].scatter(np.sort(x), np.sort(z), s=5); ax[1, 0].plot([0, 3.5], [0, 3.5], "r--"); ax[1,
 ↪  0].set_title("Q-Q plot: X vs Z")
ax[1, 1].scatter(x, z, s=5);                   ax[1, 1].set_title("scatterplot: X vs Z")
plt.tight_layout(); plt.show()

print(f"corr(X,Y) = {np.corrcoef(x, y)[0, 1]:.3f},  Spearman(X,Y) = {stats.spearmanr(x, y)[0]:.3f}")
print(f"corr(X,Z) = {np.corrcoef(x, z)[0, 1]:.3f},  Spearman(X,Z) = {stats.spearmanr(x, z)[0]:.3f}")
```

Output:

```
corr(X,Y) = 0.023,  Spearman(X,Y) = 0.010
corr(X,Z) = 0.752,  Spearman(X,Z) = 1.000
```



Task 4, part 1(c): X, Y i.i.d. $\mathrm{Exp}(2)$ and $Z = \ln X + 5$ X vs Y. They are independent and identically distributed. The Q-Q plot hugs the diagonal (scatter appears only in the far tail, where order statistics are noisy). The scatterplot is a structureless cloud over the positive quadrant, and the sample correlations are essentially zero (0.02 Pearson, 0.01 Spearman).

X vs Z. Z is a deterministic, strictly increasing function of X, so the pair is completely dependent: the scatterplot is exactly the curve $z = \ln x + 5$. Because the map is increasing, the sorted Z-values equal $\ln(\text{sorted } x) + 5$, so the Q-Q plot coincides with this curve and is far from the diagonal. The distributions differ: Z lives on all of $\mathbb{R}$, is left-skewed and has mean $5 - \gamma_E - \ln 2 \approx 3.73$, against 0.5 for X. The Pearson correlation is $0.75 < 1$ because the relation is non-linear, while Spearman is exactly 1 because it is monotone.

Conclusion. (X, Y) is independent and identically distributed. (X, Z) is **dependent** and **not identically distributed**. The two properties are logically unrelated: dependence is about the joint law, identical distribution about the marginals.

Task 4, part 2: the coin game **(a)** X counts successes in 3 independent trials with probability $\frac{1}{2}$ each, so $X \sim \text{Binomial}(3, \frac{1}{2})$, with pmf $(\frac{1}{8}, \frac{3}{8}, \frac{3}{8}, \frac{1}{8})$. It is not normal (finite support), not Poisson (bounded by 3) and not uniform.

```
# exact moments from the pmf of X ~ Binomial(3, 1/2)
v = np.arange(4); pmf = binom.pmf(v, 3, 0.5)
EX, VarX = (v * pmf).sum(), (v**2 * pmf).sum() - (v * pmf).sum()**2
EY = (v**2 * pmf).sum()
VarY = (v**4 * pmf).sum() - EY**2
print(f"E X = {EX:.2f}, Var X = {VarX:.2f},  E Y = {EY:.2f}, Var Y = {VarY:.2f}  (friend's guess (E X)^2 =
 ↪  {EX**2:.2f})")

# 100 simulated rounds
x = np.random.binomial(3, 0.5, size=100)
y = x**2
print(f"X: mean = {x.mean():.3f}, s^2 = {x.var(ddof=1):.3f}")
print(f"Y: mean = {y.mean():.3f}, s^2 = {y.var(ddof=1):.3f}")
eps = z975 * y.std(ddof=1) / np.sqrt(100)
print(f"95% CI for E(Y): {y.mean():.3f} +- {eps:.3f} = ({y.mean() - eps:.3f}, {y.mean() + eps:.3f})")

# rounds needed to separate E(Y) = 3 from 2.9 (gap 0.1)
sd = np.sqrt(VarY)
print("eps <= 0.1 needs n =", int(np.ceil((z975 * sd / 0.1)**2)))
print("80% chance of excluding 2.9 needs n =", int(np.ceil(((z975 + norm.ppf(0.8)) * sd / 0.1)**2)))
```

Output:

```
E X = 1.50, Var X = 0.75,  E Y = 3.00, Var Y = 7.50  (friend's guess (E X)^2 = 2.25)
X: mean = 1.590, s^2 = 0.850
Y: mean = 3.370, s^2 = 8.902
95% CI for E(Y): 3.370 +- 0.585 = (2.785, 3.955)
eps <= 0.1 needs n = 2882
80% chance of excluding 2.9 needs n = 5887
```

(b) $EX = np = 1.5$ and $\text{Var}X = np(1-p) = 0.75$. The simulation of 100 rounds gives $\bar{x} = 1.590$ and $s^2 = 0.850$. Both are close to the exact values (the standard error of $\bar{x}$ is $\sqrt{0.75/100} = 0.087$, so the deviation is about one standard error).

(c) The friend's argument is wrong, and it is the same mistake as in 1(a): he computes $g(EX) = (EX)^2 = 2.25$ instead of $E\,g(X) = E(X^2)$. Since $x \mapsto x^2$ is convex, $E(X^2) \geq (EX)^2$, and the gap is exactly the variance:

$$E(Y) = E(X^2) = \text{Var}X + (EX)^2 = 0.75 + 2.25 = 3.$$

Direct check: $Y \in \{0, 1, 4, 9\}$ with probabilities $\frac{1}{8}, \frac{3}{8}, \frac{3}{8}, \frac{1}{8}$ gives $EY = \frac{0+3+12+9}{8} = 3$. The entry fee is £3, so the expected net gain is 0 and **the game is fair**. The variance is

$$\text{Var}(Y) = E(X^4) - (EY)^2 = \frac{0+3\cdot 1+3\cdot 16+81}{8} - 9 = 16.5 - 9 = 7.5, \qquad \text{sd}(Y) = 2.74.$$

The simulation gives $\bar{y} = 3.370$ and $s^2 = 8.902$, close to 3 and 7.5. The deviation 0.37 is about 1.4 standard errors ($\sqrt{7.5/100} = 0.274$), so it is no evidence of unfairness.

(d) By the CLT, the 95% confidence interval for $E(Y)$ from 100 rounds is $\bar{y} \pm 1.96\, s/\sqrt{100} = 3.370 \pm 0.585 = (2.785, 3.955)$. It contains 3, but also 2.9 and 3.5, so **the simulation alone cannot tell whether the game is fair**: the half-length 0.59 is much larger than the differences we care about, and a finite simulation can never prove an exact equality anyway. To tell the fair game from one with mean payoff 2.9 (a gap of 0.1) we need $\varepsilon \leq 0.1$:

$$n \geq \left(\frac{1.96\,\sigma_Y}{0.1}\right)^2 = \left(\frac{1.96 \cdot 2.739}{0.1}\right)^2 \approx 2882 \text{ rounds.}$$

This is only the break-even point: then ε equals the gap, so the interval excludes 2.9 only when $\bar{y} > 3$, i.e. about half of the time. For an 80% chance of excluding it, $n \geq ((1.96 + 0.84)\sigma_Y/0.1)^2 \approx 5887$ rounds.

Conclusion. The game is fair and $\text{Var}(Y) = 7.5$. Telling nearly-fair games apart by simulation costs thousands of rounds because the error shrinks only like $1/\sqrt{n}$.

General instructions/hints on implementation

1. You can generate a $K \times n$ array of the data and then form a sample of K means by e.g. using

```
s = np.random.exponential(scale=1/lam, size=(K, n)).mean(axis=1)
```

for the exponential distribution $\mathcal{E}(\lambda)$ with rate λ. Note that NumPy uses the scale (mean) $1/\lambda$, not the rate. Similarly, `np.random.poisson(mu, size=(K, n))` gives Poisson samples.

2. The e.c.d.f. F_S of the data vector `s` is

```
from scipy.stats import ecdf
Fs = ecdf(s).cdf.evaluate      # callable: Fs(t)
```

3. K must be at least 10000 to generate reliable e.c.d.f.'s of sample means.
4. `scipy.stats.norm(mu, sigma).cdf` gives the c.d.f. F_Z of $\mathcal{N}(\mu, \sigma^2)$. Note that the 2nd argument is the standard deviation σ.
5. To calculate the maximal difference between the two c.d.f.'s, take a grid of 200 points t_j within 3σ of μ and return $\max_{t_j}\{|F_S(t_j) - F_Z(t_j)|\}$:

```
t = np.linspace(mu - 3*sigma, mu + 3*sigma, 200)
np.max(np.abs(Fs(t) - Fz(t)))
```

6. To plot both c.d.f.'s on one graph, use `matplotlib.pyplot.step(t, Fs(t), where="post")` for the e.c.d.f. and `plt.plot(t, Fz(t))` for the normal c.d.f.
7. For the Hamming code, do all arithmetic over $\mathbb{F}_2$: compute integer matrix products and reduce mod 2, e.g. `c = (m @ G) % 2`, `z = (r @ H) % 2`. Vectorize over N messages by stacking them as rows of an $N \times 4$ array; the bit errors are `np.random.random((N, 7)) < p`.
8. For a Q-Q plot of two samples `x` and `y` of equal size, plot `np.sort(x)` against `np.sort(y)`; for a Q-Q plot against a theoretical distribution, use `scipy.stats.probplot`.
9. Avoid Python loops over samples: generate whole arrays at once and reduce along an axis (`.mean(axis=1)`, `.sum(axis=1)`). This is both faster and closer to the mathematical formulation.